

PROJET DATA ENGINEERING

# Olist - MongoDB & FastAPI

Concevoir, stocker et exposer des données e-commerce

---

## Objectif général

À partir d'un jeu de données e-commerce réel composé de plusieurs fichiers liés, concevoir une solution MongoDB cohérente puis mettre les données à disposition au moyen d'une API REST développée avec FastAPI.

**Le chemin technique n'est pas fourni : les choix de modélisation, d'architecture et d'API doivent être proposés et justifiés.**

**Travail en binôme. Temps de réalisation : mardi, mercredi et vendredi matin. Soutenance et démonstration : vendredi après-midi.**

## 1. Contexte de la mission

**Olist** met à disposition un jeu de données public issu d'une plateforme e-commerce brésilienne. Il contient environ 100 000 commandes réalisées entre 2016 et 2018, avec des informations sur les clients, commandes, produits, vendeurs, paiements, avis et données géographiques.

**Source du dataset :** [Brazilian E-Commerce Public Dataset by Olist - Kaggle](#)

**Le jeu de données comprend notamment les fichiers suivants :**

- olist\_customers\_dataset.csv
- olist\_orders\_dataset.csv
- olist\_order\_items\_dataset.csv
- olist\_order\_payments\_dataset.csv
- olist\_order\_reviews\_dataset.csv
- olist\_products\_dataset.csv
- olist\_sellers\_dataset.csv
- olist\_geolocation\_dataset.csv
- product\_category\_name\_translation.csv

**Important :** la structure relationnelle d'origine ne doit pas être reproduite automatiquement dans MongoDB. Vous devez analyser les usages attendus et proposer un modèle documentaire adapté.

## 2. Besoin professionnel

L'équipe Data doit mettre à disposition les données e-commerce pour de futurs services internes : applications, analyses, reporting ou outils métiers. Les fichiers CSV ne constituent pas une solution exploitable directement par ces consommateurs.

**Votre mission :** concevoir une solution permettant de stocker ces données dans MongoDB et de les exposer au travers d'une API REST fiable, documentée et utilisable par un autre développeur.

**Vous devrez notamment répondre aux questions suivantes :**

- Comment représenter les différentes entités et relations dans MongoDB ?
- Quelles données doivent être embarquées dans un même document et lesquelles doivent rester séparées ?
- Comment gérer les incohérences, valeurs manquantes et formats de données ?
- Quels usages de consultation ou d'analyse l'API doit-elle permettre ?
- Comment éviter des requêtes trop coûteuses ou des réponses trop volumineuses ?
- Quels index sont pertinents et comment démontrer leur intérêt ?
- Comment rendre le service compréhensible et réutilisable par un autre développeur ?

### 3. Contraintes du projet

- MongoDB est obligatoire pour le stockage final.
- FastAPI est obligatoire pour l'exposition des données.
- Le projet doit exploiter au minimum cinq fichiers liés du dataset. Le choix des fichiers doit être cohérent avec les usages retenus.
- La modélisation MongoDB doit être argumentée ; aucun modèle de collections n'est imposé.
- Les routes de l'API doivent être conçues à partir des besoins identifiés ; aucune liste d'endpoints n'est fournie.
- L'API doit permettre plusieurs usages de consultation et donner accès à au moins un résultat agrégé utile.
- Les résultats potentiellement volumineux doivent être maîtrisés.
- Les paramètres reçus par l'API doivent être validés.
- Les erreurs doivent être gérées avec des réponses HTTP cohérentes.
- Au moins une requête MongoDB significative doit être analysée avec explain() et améliorée par un index lorsque cela est pertinent.
- Le projet doit pouvoir être installé et exécuté par une autre personne à partir du dépôt Git.

### 4. Attendus techniques minimum

| Domaine              | Attendus                                                                                                                                                         |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Données</b>       | Analyser les fichiers, leurs relations, les types, valeurs manquantes et incohérences utiles au projet. Sélectionner les données nécessaires aux usages retenus. |
| <b>MongoDB</b>       | Proposer un modèle documentaire cohérent, réaliser un import reproductible et produire les requêtes ainsi qu'au moins deux agrégations utiles au service.        |
| <b>Performance</b>   | Analyser au moins une requête avec explain() et mettre en place un index justifié lorsque cela apporte un gain pertinent.                                        |
| <b>API REST</b>      | Développer une API FastAPI connectée à MongoDB couvrant plusieurs usages de consultation et au moins un accès à un résultat agrégé.                              |
| <b>Robustesse</b>    | Valider les paramètres, gérer les erreurs HTTP et maîtriser le volume des réponses.                                                                              |
| <b>Documentation</b> | Fournir une documentation Swagger/OpenAPI exploitable et un README permettant l'installation, le lancement et l'utilisation du projet.                           |
| <b>Qualité</b>       | Mettre en place quelques tests automatisés sur les comportements essentiels de l'API.                                                                            |
| <b>Versionnement</b> | Maintenir un dépôt Git propre. Les deux membres du binôme doivent avoir des contributions identifiables dans l'historique.                                       |

## 5. Livrables attendus

- Un dépôt Git contenant l'ensemble du projet.
- Les scripts ou procédures nécessaires à la préparation et à l'import des données.
- Le code de l'API FastAPI.
- La configuration MongoDB et les choix de modélisation documentés.
- Les requêtes / agrégations principales utilisées par le service.
- La justification d'au moins un index et une analyse de performance avec explain().
- Quelques tests automatisés portant sur les comportements essentiels de l'API.
- Un fichier requirements.txt ou équivalent.
- Un fichier .env.example sans secret si des variables d'environnement sont utilisées.
- Un README expliquant l'installation, le lancement, l'architecture et l'utilisation de l'API.
- Une présentation de vulgarisation technique préparée en binôme sur la relation entre MongoDB et FastAPI, appuyée par le projet Olist.

**Le projet doit être reproductible. La priorité est donnée à un périmètre fonctionnel cohérent, maîtrisé et démontrable plutôt qu'à une multiplication de fonctionnalités incomplètes.**

## 6. Soutenance finale

**Durée indicative : 10 à 12 minutes de présentation, suivies de questions.**

**Les deux membres du binôme participent à la présentation. Chacun doit être capable d'expliquer avec ses propres mots le rôle de MongoDB, le rôle de FastAPI et la manière dont les deux technologies communiquent.**

La présentation doit permettre de comprendre :

- MongoDB expliqué simplement : son rôle, son fonctionnement général et son intérêt ;
- FastAPI expliqué simplement : son rôle et ce qu'une API apporte entre une application et une base de données ;
- la relation MongoDB - FastAPI : le parcours d'une requête, de la demande du client jusqu'à la réponse JSON ;
- pourquoi passer par une API plutôt que donner un accès direct à MongoDB ;
- les principaux avantages et inconvénients de MongoDB ;
- les principaux avantages et inconvénients de FastAPI ;
- ce que le binôme a compris, découvert ou corrigé pendant la réalisation du projet ;
- une explication vulgarisée de l'architecture, compréhensible par une personne non spécialiste de la Data.

**Le projet Olist sert d'exemple concret pour illustrer les explications. Une démonstration courte de l'API est attendue afin de montrer visuellement le chemin FastAPI - MongoDB - réponse JSON ; la démonstration ne constitue pas le sujet principal de la soutenance.**